COS 487 / 587 Course Project Part 1: IR Rangers

Esther Greene
University of Southern Maine
Portland, Maine, USA
esther.greene@maine.edu

Courtney Jackson
University of Southern Maine
Portland, Maine, USA
courtney.jackson@maine.edu

Nathaniel Serrano
University of Southern Maine
Portland, Maine, USA
nathaniel.serrano@maine.edu

Abstract

This project presents IR-Rangers, an information retrieval system for searching a collection of arXiv research papers. The system uses text preprocessing and inverted indexing to support document retrieval at both the paper and paragraph levels. We implemented and evaluated TF-IDF and BM25 ranking approaches using relevance judgments and standard information retrieval evaluation metrics. We also developed a web-based search interface using React and FastAPI, allowing users to submit queries, view ranked research papers, explore relevant vocabulary, and receive title-based autocomplete suggestions. Through this project, we examined the effectiveness of traditional retrieval techniques and integrated indexing, ranking, evaluation, and user interface components into a functional research paper search system.

Keywords

information retrieval, BM25, TF-IDF, scientific document retrieval, passage retrieval

1 Introduction

The volume of scientific literature makes finding relevant work increasingly difficult, both at the level of whole papers and of the specific passages that answer a question. This paper describes the first phase of our retrieval system for arXiv papers, which applies two traditional term-matching models, TF-IDF and BM25, to retrieving both papers and individual paragraphs. We describe our preprocessing of a noisy collection, tune BM25 per task, evaluate both models with significance testing, and analyze where term matching succeeds and where it fails, particularly on hard queries that paraphrase technical terminology. As we will detail, our findings show that BM25 significantly outperforms TF-IDF on both tasks.

The test collection consists of 34,308 arXiv papers published in August and September 2024. Each paper is provided both as raw HTML scraped from the ar5iv website and as a processed JSON file containing its identifier, title, abstract, and paragraphs. Because the JSON files were produced with rule-based extraction, they contain noise such as leftover LaTeX markup and merged words, which we discuss in Section 2.1. In total, the collection contains 2,275,574 paragraphs, which averages out to approximately 66 paragraphs per paper.

We address two tasks over this collection: *paper retrieval*, which returns papers identified by their arXiv ID (in YYMM.NNNNN format), and *paragraph retrieval*, which returns individual paragraphs. For each query, systems return their top 100 results. Three query sets are provided: Train (2,000 queries), Study (250 queries with relevance judgments, which we use for all analysis in this paper), and Test (250 queries whose judgments are withheld). Each set contains 30% easy, 50% medium, and 20% hard queries. Relevance is graded on a three-point scale: a document that fully addresses the query is relevant (2), one that provides useful information without fully addressing it is partially relevant (1), and any document not in the judgments is considered non-relevant.

2 Methodology

Our system processes both retrieval tasks with a single pipeline. Documents and queries are first cleaned and tokenized (Section 2.1); the processed text is used to build two collections, one of papers and one of paragraphs (Section 2.2), each with its own inverted index. Two ranking models, TF-IDF (Section 2.3) and BM25 (Section 2.4), score documents from these indexes; both use identical preprocessing and the same indexes, so differences between them reflect only their scoring functions. Section 2.5 describes our evaluation setup, and Section 2.6 the search interface.

Throughout, N denotes the number of documents in a collection, df_t the number of documents containing term t, $tf_{t,d}$ the number of occurrences of t in document d, and L_d and L_{avg} the length of d and the average document length, in tokens.

2.1 Preprocessing

Documents and queries pass through the same preprocessing pipeline, so that query terms match the terms stored in the index. All settings are read from a YAML configuration file, and a copy of the configuration is saved with the processed collections; queries are tokenized with that saved copy at search time. The final pipeline applies the following steps in order:

(1) **Text cleaning.** Inline mathematics delimited by dollar signs, remaining LaTeX commands (e.g., `\mathcal`), URLs, and numeric citation markers such as "[5, 6, 7]" are each replaced with a space.
(2) **Lowercasing.**
(3) **Tokenization.** Text is split into runs of letters and digits. Underscores act as separators, so a subscript such as `x_i` yields `x` and `i`.
(4) **Token filtering.** Tokens shorter than two characters and tokens consisting only of digits are discarded.
(5) **Stopword removal.** Stopwords are frequent function words, such as "the" and "and", that say little about a document's topic. We use NLTK's English list [2], extended with terms common in scientific writing: "et", "al", "fig", "figure", "table", "eq", and "section".
(6) **Stemming.** Remaining tokens are reduced to stems with the Snowball English stemmer [6], so that forms such as "model" and "models" match.

The cleaning and filtering steps were motivated by corpus statistics. With only lowercasing and stopword removal, the 50 most frequent tokens were dominated by mathematical notation rather

Table 1: Paragraph collection statistics under three preprocessing settings, defined in Section 2.1.

	Baseline	Cleaned	+ Snowball
Total tokens	167,965,275	114,907,087	114,907,087
Unique terms	781,405	659,861	589,251
Terms appearing once	367,415	297,672	272,983
Median length	58	40	40
90th pct. length	143	104	104
Longest paragraph	32,070	20,818	20,818
Over 1,000 tokens	868	117	117
Empty paragraphs	429	8,389	8,389
Under 10 tokens	82,390	230,782	230,782

than words: digits ("1", "2", "0"), single-letter variables ("n", "x", "k"), underscores from subscripts, and LaTeX commands such as "mathcal" and "mathbb". Adding these to the stopword list would also remove real words such as "text" and "prime", so we instead remove inline math as a whole. A lone dollar sign, such as in a price, would be paired with a math delimiter and the text between them deleted, so math spans are only removed from paragraphs with an even number of dollar signs. Only 1,458 paragraphs (0.06%) contain an unpaired dollar sign. They keep their math spans, although LaTeX command removal and the token filters still remove most of the resulting tokens. We evaluate the effects of our choice to employ inline math removal and stemming on retrieval in Section 4.3.

Some extraction noise is not addressed by preprocessing. Adjacent words are occasionally merged (e.g., "andenthusiasts", "inmedical"), and this also occurs in queries: Test query 2251 contains "sustainedvowel". Such tokens cannot match their component words. Display equations are absent from the extracted text altogether, and the `abstract` field contains more merged words than the copy of the abstract that appears as each paper's first paragraph (Section 2.2).

Table 1 compares three settings. *Baseline* applies only lowercasing, tokenization on letters, digits, and underscores, and stopword removal (with "not" and "no" retained). *Cleaned* applies steps 1–5 and no longer retains "not" and "no": bag-of-words models cannot use them to express negation, and "not" alone accounted for over 500,000 tokens. *+ Snowball* is the final pipeline. Cleaning reduced the collection from 168.0 million to 114.9 million tokens, a 31.6% decrease, and all of the 50 most frequent tokens became words, such as "model", "data", and "using". It also reduced the number of paragraphs longer than 1,000 tokens from 868 to 117; the longest Baseline paragraphs were collaboration author lists made up largely of LaTeX markup and ORCID identifiers. Stemming leaves the number of tokens unchanged but shrinks the vocabulary by a further 10.7%, from 659,861 to 589,251 terms. The cost is that some distinct words share a stem: "different" and "difference" both become "differ", and "generate", "generation", and "generative" all become "generat". Table 1 also shows that the number of paragraphs under 10 tokens nearly triples (82,390 → 230,782). This is because the cleaning process employed in the latter two pipelines strips math out of math-heavy paragraphs and leaves only their prose, causing many to become short fragments. Finally, cleaning removes every token from 8,389 paragraphs. Of these, 94% contain inline math or LaTeX commands: they are standalone formulas, sometimes with a few connecting words ("if", "then", "where") that are themselves stopwords. We describe how these paragraphs are handled in Section 2.2.

2.2 Document Representation

We construct two separate document collections from the processed JSONL files, namely one for paper retrieval and another for paragraph retrieval. In the paper collection, each document consists of its title and all non-empty paragraphs. This means we omit the separate abstract field. We did this because the first paragraph typically contains the abstract and usually has fewer extraction errors, such as merged words. This helps avoid duplicating the abstract within the same document.

For paragraph retrieval, we treat each non-empty paragraph as an individual document. Of the 2,275,574 original paragraphs, 8,389 become empty after preprocessing, so we exclude them. This leaves us with 2,267,185 indexed paragraphs. We keep the first paragraph of each paper because abstracts can be relevant to retrieval results, as demonstrated by paragraph 2409.12643_1 in the Train relevance judgments.

Essentially, both collections use inverted indexes that map each term to a postings list containing document numbers and term frequencies. The indexes also store individual document lengths and average document length, supporting the retrieval models described in Sections 2.3 and 2.4. Each index is built from its processed JSONL collection and saved as a JSON file. This means subsequent runs can load the existing index without rebuilding it. The paper and paragraph indexes occupy 242 MB and 974 MB on disk, with loading times of approximately 9.2 and 29.0 seconds (Table 7).

2.3 TF-IDF

TF-IDF [5, Ch. 6] weights a term in a document by how often it occurs there and how rare it is across the collection, so that frequent but uninformative terms count for little. We rank documents by the cosine similarity between TF-IDF-weighted query and document vectors.

2.3.1 Scoring Function. The weight of term t in document d uses logarithmic term frequency and base-10 logarithms:

$$w_{t,d} = \left(1 + \log_{10} tf_{t,d}\right) \cdot idf_t, \qquad idf_t = \log_{10} \frac{N}{df_t}. \tag{1}$$

Query weights wt,q are computed the same way from the number of times t occurs in the query. A document's score is the cosine of the angle between the two weight vectors:

$$\text{score}(q,d) = \frac{\sum_{t \in q \cap d} w_{t,q}\, w_{t,d}}{\|\mathbf{w}_q\|\, \|\mathbf{w}_d\|}, \qquad \|\mathbf{w}_d\| = \sqrt{\textstyle\sum_{t \in d} w_{t,d}^2}. \tag{2}$$

The logarithm makes term frequency sublinear: ten occurrences of a term weigh about twice as much as one, not ten times as much. IDF is never negative; a term that occurs in every document has an IDF of zero and is skipped. Because the document norm sums over *all* of a document's terms, documents with many distinct terms have larger norms, which dilutes the weight of their matching terms. The query norm is the same for every document and does not affect the ranking.

2.3.2 Implementation. TF-IDF uses the same inverted indexes as BM25 (Section 2.2). When the model is created, it computes the IDF of every term and the norm of every document in a single pass over all postings. At query time, the query, preprocessed as in Section 2.1, is weighted as above, skipping terms that are absent from the index or have zero IDF. For each remaining term, a Python loop walks its postings and accumulates wt,qwt,d for each document; the sums are then divided by the norms. The top 100 documents are selected with a heap, with ties broken by internal document number, as for BM25.

2.4 BM25

BM25 addresses two weaknesses of the TF-IDF weighting in Section 2.3: it saturates the contribution of repeated terms, and it normalizes document length relative to the collection average rather than by vector length. We describe the scoring function we use, our implementation, and how we tuned its two parameters, k_1 and b, for each collection.

2.4.1 Scoring Function. BM25 scores a document d for a query q as

$$\text{score}(q,d) = \sum_{t \in q} idf_t \cdot \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,L_d/L_{avg}\right)} \tag{3}$$

$$idf_t = \ln\left(1 + \frac{N - df_t + 0.5}{df_t + 0.5}\right) \tag{4}$$

where the sum runs over query tokens, so a term repeated in the query contributes once per occurrence. The term-frequency component saturates: as $tf_{t,d}$ grows, it approaches $k_1 + 1$, so additional occurrences of a term add progressively less. The parameter k_1 controls how quickly this happens; a small k_1 makes the score depend mainly on whether the term occurs at all, while a large k_1 lets repetition keep contributing. The parameter $b \in [0, 1]$ controls length normalization: a document longer than average ($L_d > L_{avg}$) has a larger denominator and therefore a lower score for the same term frequency, with $b = 0$ disabling normalization and $b = 1$ applying it fully.

We use the "Lucene (accurate)" variant of BM25 described by Kamphuis et al. [4], the third row of their Table 1. Since they found no significant differences in effectiveness among eight BM25 variants, we chose on practical grounds. First, its IDF adds one inside the logarithm. The original formulation [7] assigns negative weight to terms that occur in more than half the documents, so matching such a term would lower a document's score, and a document matching more query terms could score lower than one matching fewer. In our paper collection, 312 terms exceed this threshold, including words common in our natural-language queries such as "use", "result", and "differ" (as in "How does X differ from Y?"). Second, it uses exact document lengths, which our index already stores, rather than Lucene's compressed approximation, which Kamphuis et al. found gave negligible speed benefits. Third, variants designed for long documents, such as BM25L and BM25+, add a further parameter to tune without significant gains in their study. Following Robertson et al. [7], we also multiply the term-frequency component by $k_1 + 1$; this scales all scores by a constant and does not change the ranking.

2.4.2 Implementation. Our implementation scores only documents that contain at least one query term, accumulating each term's contribution over its postings (term-at-a-time). Two quantities are precomputed: idf_t for every term when the model is created, and the length-normalization term $k_1(1-b+b\,L_d/L_{avg})$ for every document, which is recomputed only when k_1 or b change. A term repeated in the query is processed once, with its contribution multiplied by its query frequency. After scoring, the top 100 documents are selected, with ties broken by internal document number so that runs are reproducible.

Our first implementation accumulated scores in a Python loop over individual postings. It was correct, but averaged 91 ms per query on papers and 1.08 s per query on paragraphs, so a full paragraph tuning grid (48 parameter settings for 1,600 queries) would have required roughly a day of computation. We therefore vectorized scoring with NumPy, so that only the loop over query terms remains in Python. Each term's postings (its document numbers and term frequencies) are converted to arrays once and cached; the term's contributions to all documents in its postings are then computed in one array operation and added to a dense score array indexed by document number. The top 100 are found by partial selection rather than a full sort. This reduced average search time to 15 ms on papers and 105 ms on paragraphs, speedups of 6× and 10× (Section 3.4), and the full paper grid search ran in about eight minutes. The vectorized version produced the exact same evaluation metrics as the original on our tuning queries over the full index.

2.4.3 Parameter Tuning. We tuned k_1 and b separately for each collection on the Train queries, never using Study queries. All 2,000 Train queries were shuffled with a fixed seed and split into 1,600 tuning queries and 400 held-out queries. On the tuning queries, we evaluated every setting in a grid: $k_1 \in \{0.9, 1.2, 1.8, 2.4, 3, 4, 5, 6, 8\}$ for papers and $k_1 \in \{0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.5, 1.8\}$ for paragraphs, each with $b \in \{0.3, 0.5, 0.6, 0.75, 0.9, 1.0\}$. We selected the setting with the highest nDCG@5. Among our evaluation metrics, nDCG@5 is the only one that both uses graded relevance and focuses on the top of the ranking, which is what users of the interface see first; deeper metrics such as P@10 are less informative here, because most queries have only a few judged documents.

The selected settings are $k_1 = 4.0$, $b = 0.75$ for papers and $k_1 = 0.4$, $b = 0.9$ for paragraphs (Figures 1a and 1b). The tenfold difference in k_1 likely reflects document length. Papers average about 3,361 tokens, so a term repeated many times is genuinely informative about a paper's topic, and a large k_1 lets repetition keep contributing. Paragraphs have a median length of about 40 tokens and rarely repeat a term, so a small k_1, which makes the score depend more on whether a term occurs, works best. Both collections favor substantial length normalization: $b = 0.3$ is the worst value in every row of the paper grid. Paragraphs, which prefer stronger normalization ($b = 0.9$), also vary more in length relative to their mean than papers do (coefficient of variation 0.95 vs. 0.69), partly because of a long tail of very long paragraphs produced by extraction errors (Section 2.1), which would otherwise match many query terms simply by length. The paper optimum also lies on a broad plateau ($k_1 \geq 3$, $0.5 \leq b \leq 0.9$) in which scores differ by about 0.01 or less, so the exact choice within it matters little.

For both collections, the tuned settings outperformed on the 400 held-out queries both the common textbook values ($k_1 = 1.2$,

Table 2: nDCG@5 on 400 held-out Train queries not used for tuning.

	Selected (k_1, b)	Tuned	Textbook [5] (1.2, 0.75)	Default (0.9, 0.4)
Papers	(4.0, 0.75)	**0.473**	0.454	0.436
Paragraphs	(0.4, 0.9)	**0.371**	0.354	0.344

$b = 0.75$) [5, Sec. 11.4.3] and the default of the Anserini toolkit [8] ($k_1 = 0.9$, $b = 0.4$), as used by Kamphuis et al. [4]. (Table 2), so the selected parameters generalize beyond the queries used to choose them. The choice is also robust to the target metric: tuning for nDCG or MRR instead selected the same setting for papers, and tuning for MRR on a separate sample of 1,000 Train queries selected the same setting for paragraphs. We compare tuned and default parameters on the Study queries in Section 3.2.

2.5 Evaluation Setup

We evaluate each run's top 100 results with five metrics: nDCG, nDCG@5, MRR, P@5, and P@10, computed with ranx [1]. For a ranking with relevance grades $rel_i \in \{0, 1, 2\}$ at ranks $i = 1, 2, \ldots,$

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{rel_i}{\log_2(i+1)}, \qquad \mathrm{nDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}, \quad (5)$$

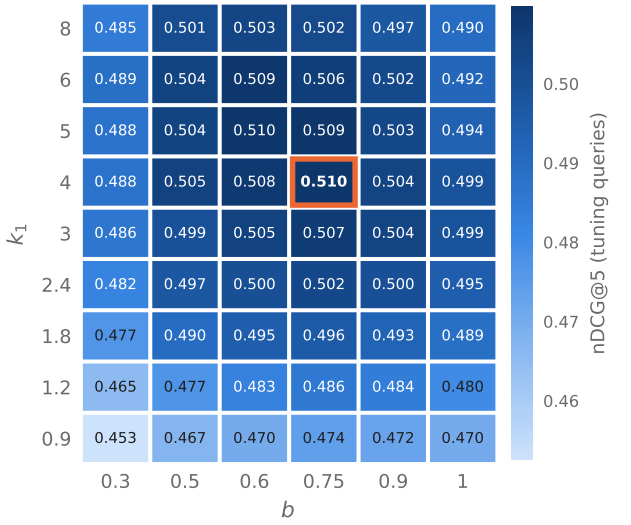
where IDCG@k is the DCG of an ideal ranking of the judged documents [3]; nDCG without a cutoff uses the full top 100. MRR is the mean over queries of 1/r, where r is the rank of the first relevant document (0 if none is retrieved), and P@k is the fraction of the top k documents that are relevant. Documents absent from the judgments count as non-relevant.

nDCG uses the relevance grades directly as gains. For MRR and P@k, we report two variants: a lenient one, in which documents of grade 1 or 2 count as relevant, and, for MRR and P@5, a strict one, in which only grade-2 documents do. Queries without any grade-2 document are excluded from the strict metrics, which are therefore averaged over 238 of the 250 Study queries.

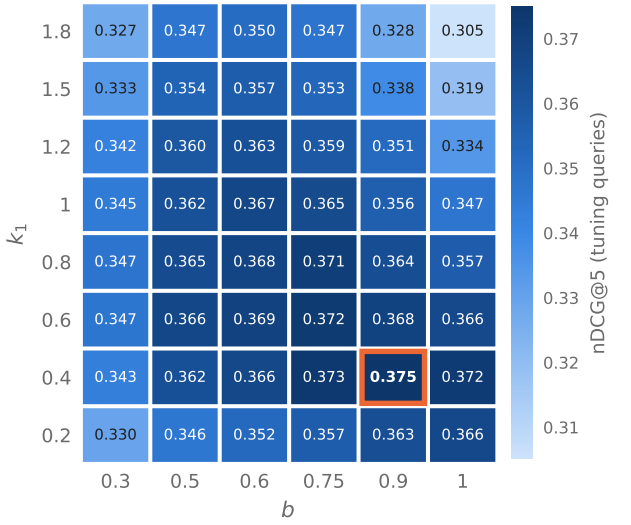
We test differences between runs with a two-sided paired Student's t-test over queries at p<0.05, using ranx. We apply it to BM25 versus TF-IDF on each collection, to tuned versus default BM25 parameters, and to the preprocessing settings compared in Section 4.3. The three query sets serve distinct purposes: Train queries are used only for tuning BM25 (Section 2.4), Study queries for all reported results, and the Test queries, whose judgments are withheld, for our submitted runs and manual annotation (Section 3.3).

2.6 Search Interface

We developed a web-based search interface that lets users interact with the paper retrieval system. The interface uses a React frontend connected to a Python FastAPI through HTTP endpoints. At startup, the backend loads the previously constructed paper index and then initializes the BM25 retrieval model using the tuned parameters described in Section 2.4 ($k_1 = 4.0$, $b = 0.75$). Search queries use the same preprocessing configuration as the indexed documents. We did this to try to keep consistency between query terms and index



(a) Papers. The selected setting ($k_1 = 4.0$, $b = 0.75$) lies within a broad plateau ($k_1 \geq 3$, $0.5 \leq b \leq 0.9$) in which settings differ by about 0.01 or less.



(b) Paragraphs. The region within about 0.01 of the selected setting ($k_1 = 0.4$, $b = 0.9$) is narrower ($0.4 \leq k_1 \leq 0.6$, $0.75 \leq b \leq 1.0$), and scores fall off sharply for large k_1 combined with strong length normalization.

Figure 1: BM25 nDCG@5 on 1,600 Train tuning queries. Rows and columns are the tested values, not to scale; scores are rounded to three decimals. Outlined cells mark the selected settings, which scored highest before rounding.

vocabulary. The backend then retrieves and returns the top-ranked papers, which are seen in the interface.

To present search results, we generate a separate metadata file which has paper identifiers, titles, publication information, and

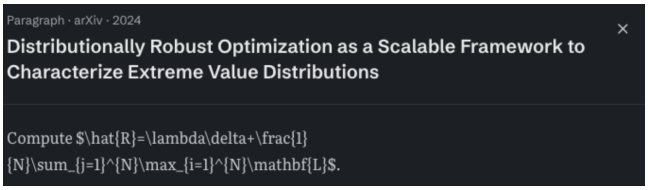


Figure 2: Overview of the document preview.

abstracts. Because the original abstract fields can contain extraction problems, the metadata generation process looks to prioritize the first paragraph of each paper. This is because we generally see a cleaner version of the abstract. Search results display paper titles alongside excerpts selected from their abstracts. So rather than truncating abstracts at a fixed character position, our backend identifies sentences containing query terms to provide excerpts that better reflect the relevance of each result.

2.6.1 Interface Features. Our interface supports a variety of features including: vocabulary highlighting, title autocomplete, document previewing, document bookmarking, search history, and settings.

The search feature works in conjunction with vocabulary highlighting and title autocomplete Query terms are displayed in their original readable forms, excluding stopwords. We did this in regard to stems used internally for retrieval. Matching terms are highlighted in (yellow) result excerpts, and users can toggle individual highlights. As a user types, autocomplete suggests paper titles containing the entered text. To reduce repeated string processing, we cache the lowercase versions of paper titles when the backend starts. These features are exposed through the /api/search, /api/vocabulary, and /api/suggest endpoints. This is in addition to the /api/health endpoint used to verify backend availability.

Users can bookmark retrieved documents to customized, named collections for future reference, with each collection supporting up to 100 documents. From the dedicated Collections page, users can manage their saved items, allowing for the easy removal of individual documents or entire collection folders.

Additionally, an interactive document preview capability is accessible directly from both the Search Results and Collections pages, enabling quick inline inspection of document content.

The system maintains a comprehensive record of previous search queries. Similar to collection management, users retain full control over their data and can clear their history by individual query, by specific dates, or in its entirety.

3 Evaluation Results

We report results on the 250 Study queries, using the metrics and significance test described in Section 2.5. We first compare TF-IDF and BM25 overall and by query difficulty (Section 3.1), then measure the effect of tuning BM25 (Section 3.2), summarize our manual judgments of the Test queries (Section 3.3), and report efficiency (Section 3.4). Explanations of these results are deferred to Section 4.

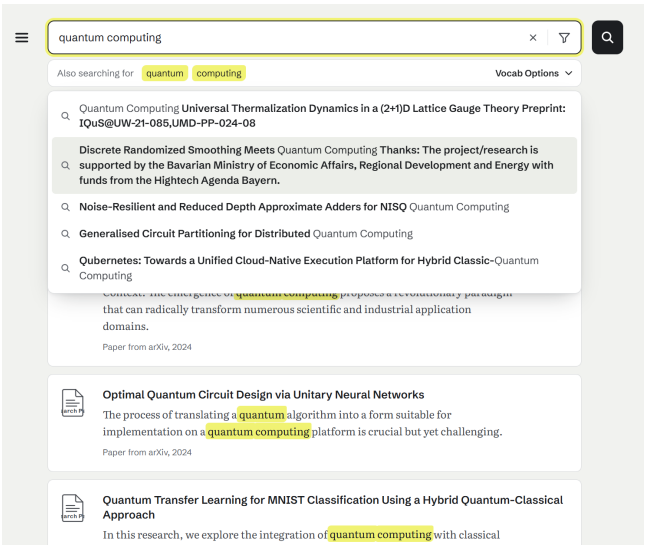


Figure 3: IR-Rangers search interface displaying ranked research papers and highlighted query terms.

Table 3: Effectiveness on the 250 Study queries. Strict metrics count only fully relevant (grade 2) documents and are averaged over the 238 queries that have one. * marks a significant improvement of BM25 over TF-IDF on the same collection (paired t-test, $p < 0.05$).

	Papers		Paragraphs	
	TF-IDF	BM25	TF-IDF	BM25
nDCG	0.397	**0.582***	0.301	**0.474***
nDCG@5	0.313	**0.514***	0.229	**0.416***
MRR	0.332	**0.544***	0.298	**0.516***
P@5	0.094	**0.137***	0.103	**0.174***
P@10	0.058	**0.080***	0.067	**0.107***
MRR (strict)	0.323	**0.535***	0.249	**0.450***
P@5 (strict)	0.085	**0.124***	0.070	**0.115***

Table 4: nDCG@5 on Study queries by difficulty level.

	Easy (75)	Medium (125)	Hard (50)
Papers, TF-IDF	0.399	0.331	0.137
Papers, BM25	0.635	0.571	0.187
Paragraphs, TF-IDF	0.375	0.227	0.015
Paragraphs, BM25	0.676	0.425	0.003

3.1 Effectiveness

BM25 significantly outperformed TF-IDF on every metric for both tasks (Table 3), improving nDCG@5 by 64% on papers and 82% on paragraphs. Absolute precision is low for both models, as most Study queries have only one to five judged documents, which caps P@10 well below 1. We analyze the sources of BM25's advantage in Section 4.

Table 5: BM25 with tuned parameters (Section 2.4) versus the Anserini default ($k_1 = 0.9$, $b = 0.4$) on Study queries. * marks a significant difference (paired t-test, $p < 0.05$); per-difficulty columns are nDCG@5 and were not tested separately.

				nDCG@5 by difficulty		
	nDCG	MRR	nDCG@5	Easy	Med.	Hard
Papers, default	0.542	0.501	0.489	0.649	0.514	0.185
Papers, tuned	0.582*	0.544*	0.514	0.635	0.571	0.187
Paragraphs, default	0.457	0.487	0.394	0.684	0.378	0.000
Paragraphs, tuned	0.474*	0.516*	0.416*	0.676	0.425	0.003

Table 6: Our judgments of BM25's top-ranked paragraph for the 250 Test queries.

	Grade 0	Grade 1	Grade 2	Relevant
Easy (75)	5	21	49	93.3%
Medium (125)	20	57	48	84.0%
Hard (50)	28	15	7	44.0%
All (250)	53	93	104	78.8%

Effectiveness falls sharply with query difficulty for both models (Table 4). On hard queries, BM25's advantage over TF-IDF on papers narrows (nDCG@5 0.187 vs. 0.137), and on paragraphs, neither model retrieves a judged relevant result for almost any hard query. As Section 3.3 shows, this partly reflects incomplete judgments rather than retrieval failure alone.

3.2 Effect of Tuning

Tuning BM25's parameters significantly improved nDCG and MRR on both collections, and nDCG@5 on paragraphs (Table 5). On papers, the nDCG@5 gain was not significant. The improvements came almost entirely from medium-difficulty queries: easy queries were already ranked well under the default parameters, and hard queries failed under both.

3.3 Test Annotation

We judged the top-ranked BM25 paragraph for each of the 250 Test queries on the three-point scale (Table 6). Annotators judged the paragraph alone, using the paper title only as context. The top-ranked paragraph was judged relevant (grade 1 or 2) for 78.8% of queries and fully relevant for 41.6%, falling from 93.3% of easy queries to 44.0% of hard queries.

These rates are much higher than BM25's P@1 of 0.452 on Study paragraphs according to the official judgments, but most of the difference is explained by incomplete judgments rather than by disagreement. In Study, BM25's top-ranked paragraph was unjudged, and therefore counted as non-relevant, for 55% of queries overall and for all 50 hard queries, whereas our annotation judged each retrieved paragraph directly. Where judgments exist, the two agree closely: the top-ranked paragraph is fully relevant for 35.2% of Study queries and 41.6% of Test queries in our judgments, with most of the remaining gap in partial relevance. Study scores for paragraph retrieval, and especially near-zero scores on hard queries, should therefore be read as lower bounds.

Table 7: Indexing, startup, and query times, and index sizes. Query times are averages over the 250 Study queries (pure-Python BM25: 50 queries) and exclude query tokenization; startup includes loading the index and each model's precomputation. Measured single-threaded on a shared server, so times are approximate.

	Papers	Paragraphs
Documents	34,308	2,267,185
Avg. document length (tokens)	3,361	50.7
Indexing (build and save)	66 s	182 s
Index size on disk (JSON)	242 MB	974 MB
Startup, BM25	9.4 s	28.7 s
Startup, TF-IDF	18.1 s	68.7 s
Query, BM25 (NumPy)	9.2 ms	74.3 ms
Query, BM25 (pure Python)	91 ms	1,076 ms
Query, TF-IDF (pure Python)	95.6 ms	1,115.5 ms

3.4 Efficiency

All times were measured single-threaded with Python 3.8 on a shared server with an Intel Xeon Gold 5215 CPU (2.50 GHz) and 62 GiB of RAM. Preprocessing the 34,308 raw papers into both collections takes about 4.5 minutes, after which building both indexes takes about 4 minutes (Table 7).

Building the paragraph index takes about three times as long as the paper index, although both collections contain a similar number of tokens, because the paragraph index records many more postings: each term is stored once per paragraph in which it occurs rather than once per paper. The same effect governs query time. Paragraph queries take longer than paper queries despite much shorter documents, because the postings of a common query term span far more documents (up to 2.27 million rather than 34 thousand), and every posting must be scored.

The gap between BM25 and TF-IDF reflects our implementations rather than the models themselves. TF-IDF uses the same pure-Python loop over postings as our original BM25, and the two run at nearly the same speed (95.6 vs. 91 ms on papers; 1,115.5 vs. 1,076 ms on paragraphs), so vectorizing TF-IDF would likely yield a similar speedup of roughly an order of magnitude. TF-IDF also has a longer startup, since it computes a vector norm for every document by walking all postings before the first query; for paragraphs this adds about 40 seconds.

Our indexes are stored as JSON, which is simple but costly to load: the 974 MB paragraph index occupies about 5.5 GB of memory during retrieval, because each posting is stored as Python objects, and loading accounts for most of the startup time. Storing postings as compact numeric arrays would reduce both memory use and startup time.

4 Results Analysis

4.1 Why BM25 Outperforms TF-IDF

Because both models share the same preprocessing pipeline and indexes, their differences come entirely from how they weight and combine terms. Three properties of the scoring functions are relevant. First, cosine-normalized TF-IDF includes the IDF in both the query weight and the document weight, so a matching term contributes roughly in proportion to idf_t^2, and rare terms tend to dominate since the ratio of IDF values between them and more common terms gets amplified. For example, "sam" has 4.6 times the IDF of "view" in our paper collection, but contributes about 21 times as much once squared. BM25 counts IDF once. Second, TF-IDF's term frequency weight $1 + \log tf$ grows without bound, while BM25's saturates at a rate set by k_1. Third, TF-IDF normalizes by the length of the document's weight vector, which grows with the number of distinct terms, while BM25 normalizes relative to the average document length, with a strength set by b.

BM25's advantage does not come from tuning: with Anserini's default parameters ($k_1 = 0.9$, $b = 0.4$; [8]) it still outperformed TF-IDF on both collections (nDCG 0.542 vs. 0.397 on papers, 0.457 vs. 0.301 on paragraphs; Table 5), and tuning added only about 0.02 to 0.04 on top of that. Nor is it an artifact of our preprocessing: TF-IDF on papers does best without stemming (Section 4.3), yet even then it reaches nDCG@5 of 0.379, against 0.542 for BM25 in the same configuration.

Query 2007 shows how squared IDF weighting can hurt TF-IDF: *"How does the performance of multi-view mask learning compare to the masks generated by SAM from a single 2D image?"* The relevant paper (3,017 tokens) matches every query term, including "multi", "view", and "compare". TF-IDF instead ranks first a short paper on SAM itself (934 tokens) that lacks those terms. Because "sam" is the rarest query term ($df = 734$), that single term contributes 0.167 to the SAM paper's TF-IDF score, more than the relevant paper's entire score (0.135). In BM25, "sam" counts once and saturates, so 46 occurrences are worth only slightly more than 14 (18.5 vs. 15.2), and the relevant paper's coverage of the remaining terms ranks it first.

The same mechanisms can also favor TF-IDF. For query 2112, *"What is the reformulated wave function for Cooper pairs in the BCS ground state for spin-orbit-coupled Fermi gases?"*, TF-IDF ranks the relevant paper first while BM25 places it third. Both candidates contain every query term, but BM25's top result is a 6,356-token paper that repeats "Fermi" 79 times and "gases" 18 times. With $k_1 = 4.0$, BM25's term weight saturates slowly, so this repetition outweighs the relevant paper's stronger match on the most specific term, "BCS" (26 vs. 12 occurrences). TF-IDF's logarithmic term frequency dampens the repetition, its squared IDF emphasizes "BCS", and its cosine normalization penalizes the longer paper. The large k_1 chosen for papers is best on average, but this query shows its cost.

BM25's relative advantage is larger on paragraphs than on papers (nDCG@5 +82% vs. +64%), contrary to what one might expect if long documents were the main obstacle for TF-IDF. A likely contributor is that cosine normalization favors very short documents: a paragraph of a few tokens containing one query term has a high cosine score, and 230,782 paragraphs have fewer than 10 tokens after cleaning (Table 1). Squared IDF weighting also gives rare tokens, including residual extraction noise such as merged words, outsized influence.

4.2 When Term Matching Succeeds and Fails

Both models succeed when a query contains distinctive terms that also appear in the relevant documents, typically the name of a method, dataset, or system. For query 2002 ("How can C-Uniform trajectories be created using random-walks and optimization methods?"), BM25 ranks first the paper titled "C-Uniform Trajectory Sampling For Fast Motion Planning", and queries naming systems such as O-RAN or SAM behave similarly. Easy queries, which tend to reuse the vocabulary of their source documents, reach nDCG@5 of 0.635 (papers) and 0.676 (paragraphs) with BM25.

Effectiveness falls sharply on hard queries (Table 4), for two reasons. The first is vocabulary mismatch: hard queries often paraphrase technical terms in general language, which exact term matching cannot bridge. Query 2011 asks: "Why is it harder to identify specific heart rhythm abnormalities using wrist-worn optical sensors than with electrical cardiac monitoring, and what complicates this process in wearable devices?" Its fully relevant paragraph answers exactly this, but in technical terms ("arrhythmias", "PPG", "ECG", "smartwatch"); after stemming, it shares a single content term with the query ("rhythm"), and none of its paper's five judged paragraphs appears in BM25's top 100. BM25's top-ranked paragraph instead shares seven terms with the query ("wearable", "monitoring", "heart", "electrical", "cardiac", "abnormalities", "identify"), yet discusses identifying *individuals* from ECG signals. Paper retrieval suffers less: the relevant paper ranks 12th, because its many paragraphs together share more terms with the query than any single paragraph does. This is why hard queries hurt paragraph retrieval most.

Our Test annotations showed related failures. Shared words with different meanings, such as "epsilon" as machine precision rather than a variable, "independent" in independent sets rather than conditional independence, and "influence" in influence functions rather than outliers, produced top-ranked paragraphs from unrelated fields. Preprocessing contributes as well: queries that spell out notation ("the ratio of epsilon to kappa") lose their matching terms in documents because inline math is removed (Section 4.3), and extraction errors appear in queries too, as in Test query 2251's "sustainedvowel".

The second reason is incomplete judgments. In Study, BM25's top-ranked paragraph was unjudged for all 50 hard queries (Section 3.3), so these queries score near zero even when a retrieved paragraph is useful. In our judgments of Test queries, 44% of top-ranked paragraphs for hard queries were relevant, mostly partially. The official scores therefore understate performance on hard queries, although genuine failures such as query 2011 remain. Bridging paraphrases requires matching meaning rather than terms, which motivates the use of neural retrieval models in future work.

4.3 Preprocessing Ablation

We isolated two preprocessing choices on Study queries (Table 8). Removing inline math, which reduced the collection by 31.6%, had

Table 8: Preprocessing ablation on Study queries, nDCG@5, with BM25 parameters fixed to the values tuned for the main configuration (math removed, Snowball stemming). Comparing the first two columns isolates math removal; comparing the last two isolates stemming. * marks a significant difference from the column to its left (paired t-test, $p < 0.05$).

	Main	Math kept	Math kept, no stemming
Papers, TF-IDF	0.313	0.320	0.379*
Papers, BM25	0.514	0.534*	0.542
Paragraphs, TF-IDF	0.229	0.238	0.235
Paragraphs, BM25	0.416	0.416	0.378*

little effect on retrieval. It made no significant difference for either model on paragraphs or for TF-IDF on papers, and slightly reduced paper BM25 (nDCG@5 0.534 to 0.514, significant only for nDCG@5 and strict P@5). Papers contain enough spelled-out notation, such as "epsilon" or "kappa", for kept math tokens to provide a few additional correct matches. Math removal is a reasonable tradeoff for a 31.6% reduction in collection size.

Stemming mattered more, and in opposite directions. It significantly improved paragraph BM25 on every metric (nDCG@5 0.378 to 0.416, MRR 0.457 to 0.509), but significantly reduced paper TF-IDF on every metric (nDCG@5 0.379 to 0.320), with no significant effect on paper BM25 or paragraph TF-IDF. A plausible explanation is that short paragraphs gain from the additional matches stemming creates, while for long papers under cosine normalization, conflated stems (e.g., "generat" for "generate", "generation", and "generative") make common stems even more frequent and dilute term weights. Even without stemming, however, TF-IDF remains far below BM25 (Section 4.1)

5 Conclusion

In this project we developed IR-Rangers, an information retrieval system designed to search a collection of arXiv research papers. Our system uses text preprocessing, inverted indexing, and two retrieval approaches, TF-IDF and BM25, to rank documents based on user queries. We explored both paper-level and paragraph-level document data and evaluated retrieval effectiveness using relevance judgments and ranking metrics.

We also developed a web-based search interface using React and FastAPI. The interface lets users search for research papers, view ranked results, explore relevant vocabulary, and get title suggestions as they type. These features were designed to make the retrieval system more accessible and easier to use.

Our system demonstrates how traditional information retrieval techniques can support research paper search. Keeping this in mind, we admit there are opportunities for improvement. Future work might include exploring additional ranking methods, improving query suggestions, and expanding the search interface to support paragraph-level retrieval. Overall, this project provided practical experience in building, evaluating, and integrating the components of an information retrieval system.

References

[1] Elias Bassani. 2022. ranx: A blazing-fast python library for ranking evaluation and comparison. In *European Conference on Information Retrieval*. Springer, 259–264.

[2] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media.

[3] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems* 20, 4 (2002), 422–446.

[4] Chris Kamphuis, Arjen P. de Vries, Leonid Boytsov, and Jimmy Lin. 2020. Which BM25 Do You Mean? A Large-Scale Reproducibility Study of Scoring Variants. In *Advances in Information Retrieval: 42nd European Conference on IR Research (ECIR 2020)*. Springer, 28–34.

[5] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

[6] Martin F. Porter. 2001. Snowball: A Language for Stemming Algorithms. https://snowballstem.org/texts/introduction.html.

[7] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Gatford. 1994. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*. 109–126.

[8] Peilin Yang, Hui Fang, and Jimmy Lin. 2017. Anserini: Enabling the use of lucene for information retrieval research. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*. 1253–1256.